

INTRODUCTION TO OPERATING SYSTEM CS-222T



Introduction

- ❑ What is an operating system?
 - ❑ Have you ever used one? Examples?
- ❑ Why is it important?
- ❑ Who uses an operating system?



What is an OS?

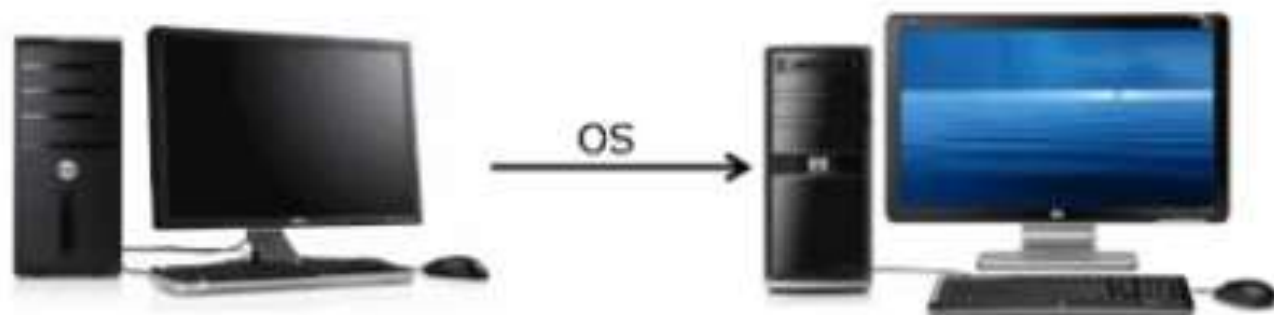
□ Have you seen one before?





Why use an OS?

- Because we need to interact with machines





OS in other Machines

□ Have you seen one before?





Other Devices





What is an Operating System?

- ❑ A program that acts as an intermediary between a user of a computer and the computer hardware
- ❑ Operating system goals:
 - ❑ Execute user programs and make solving user problems easier
 - ❑ Make the computer system convenient to use
 - ❑ Use the computer hardware in an efficient manner





Operating System Definition

- ❑ No universally accepted definition
- ❑ "The one program running at all times on the computer" is the **kernel of the operating system**.
 - ❑ Everything else is either
 - ❑ a system program (ships with the operating system) , or
 - ❑ an application program.
- ❑ OS is a **resource allocator**
 - ❑ Manages all resources
 - ❑ Decides between conflicting requests for efficient and fair resource use
- ❑ OS is a **control program**
 - ❑ Controls execution of programs to prevent errors and improper use of the computer



COMPUTER SYSTEM STRUCTURE

A computer system can be divided into four components;

1. Hardware

Hardware provides the basic computing resources for the system. e.g. Central processing unit (CPU), memory and input/output (I/O) devices.

2. Operating System

Operating system controls and coordinates the use of the hardware among the various application programs for various users.

COMPUTER SYSTEM STRUCTURE(CONT.)

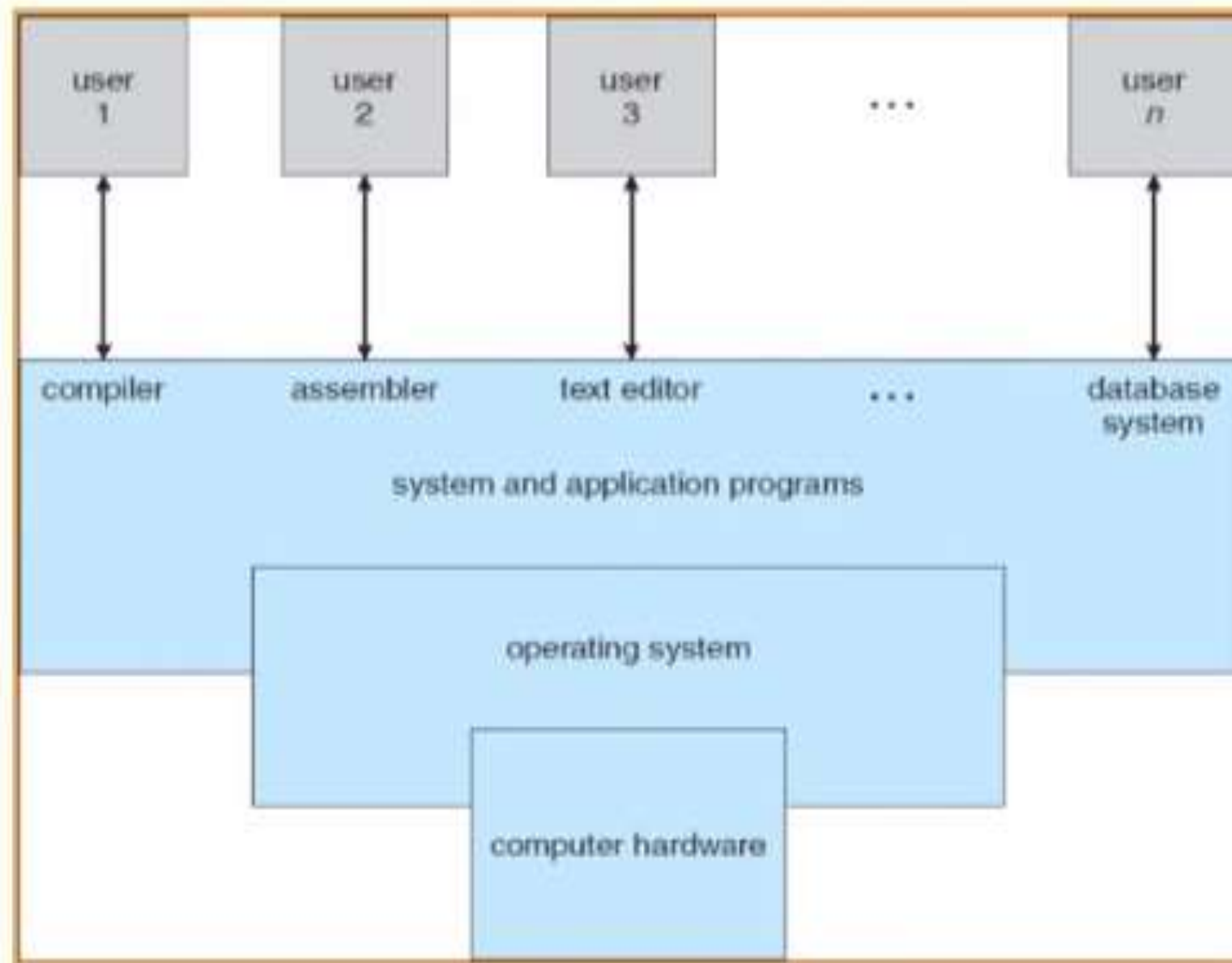
3. Application Programs

Application programs, such as word processors, spreadsheets, compilers and web browsers define the ways in which these resources are used to solve user's computing problems.

4. Users

Peoples, machines, other computers etc.

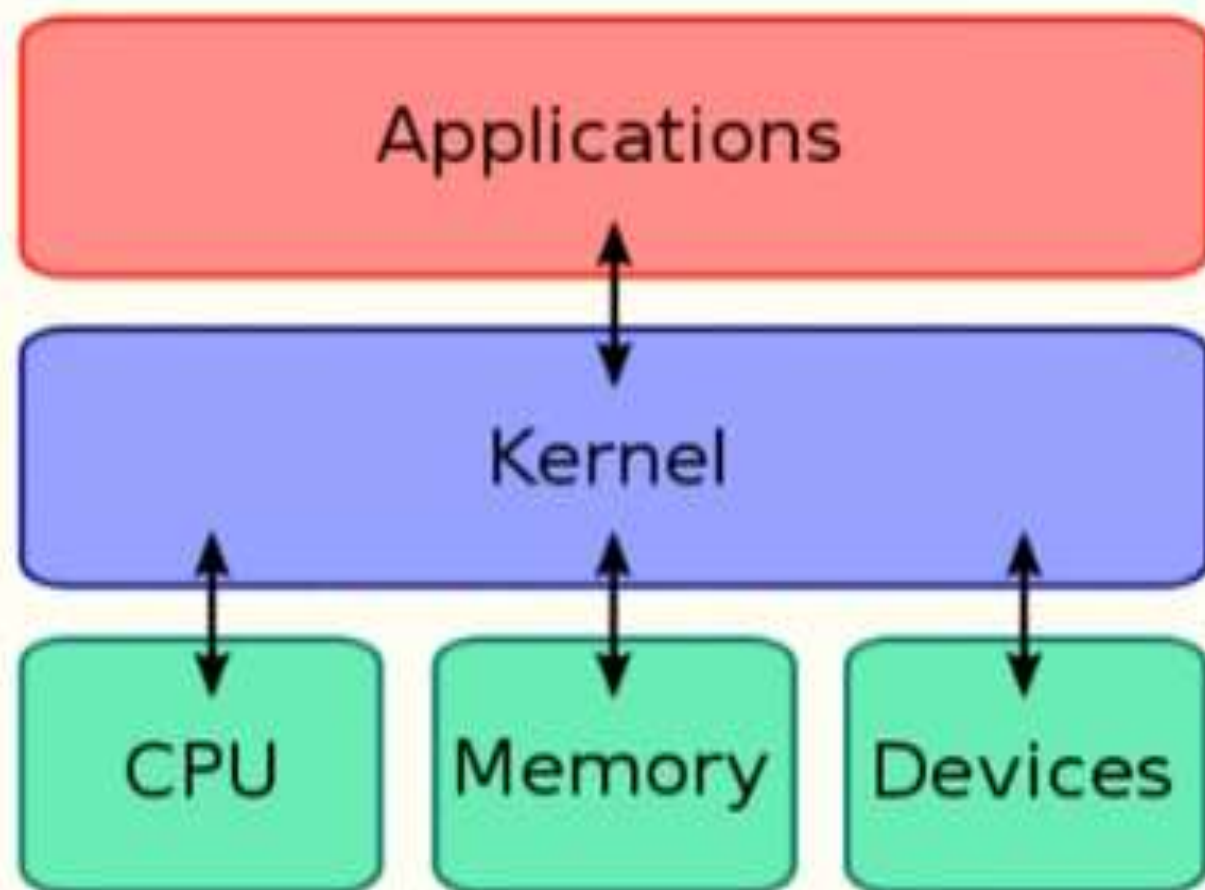
COMPUTER SYSTEM STRUCTURE(CONT.)



KERNEL

- ❑ Kernel is a **basic component** of an operating system.
- ❑ It provides the **bridge** between applications and the actual data processing done at the hardware level.
- ❑ The kernel's responsibilities include managing the system's resources (the communication between **hardware** and **software** components). In simple we can say that, kernel connects the application software to the hardware of a computer.

KERNEL (CONT.)



BOOTSTRAP

When a computer is **powered-up** or **rebooted**- it needs to have an initial program to run. This initial program is called *bootstrap*. This program is loaded at power-up or reboot.

This *bootstrap* program is typically stored in **read-only memory** (ROM) or electrically erasable programmable read-only memory (EEPROM), generally known as firmware.

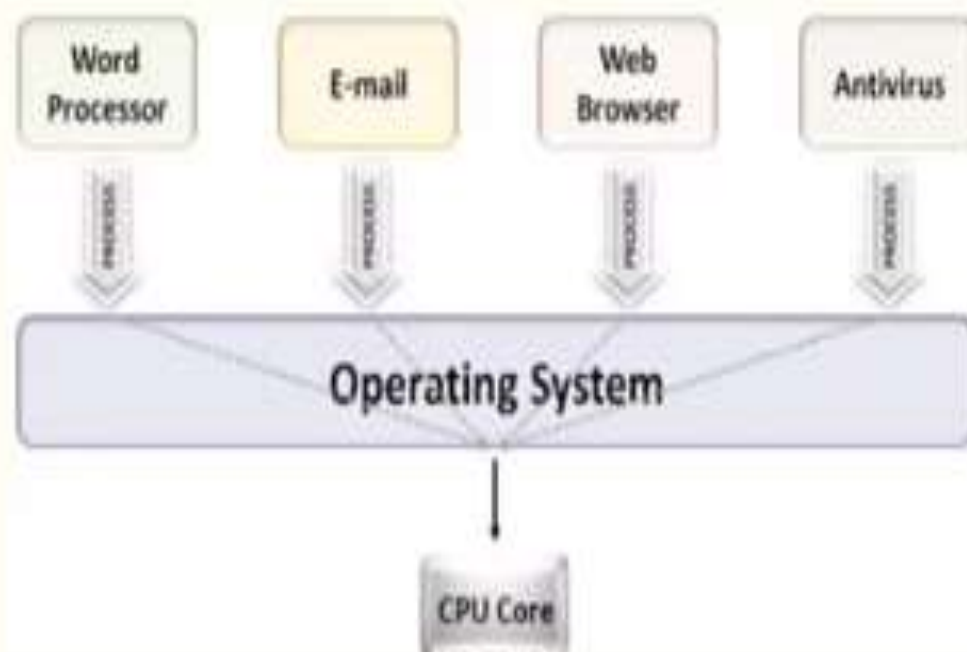
Purpose of *bootstrap* program;

- It initializes the aspects of the system.
- Loads operating system kernel and starts execution

COMPUTER SYSTEM ARCHITECTURE

Single-Processor Systems

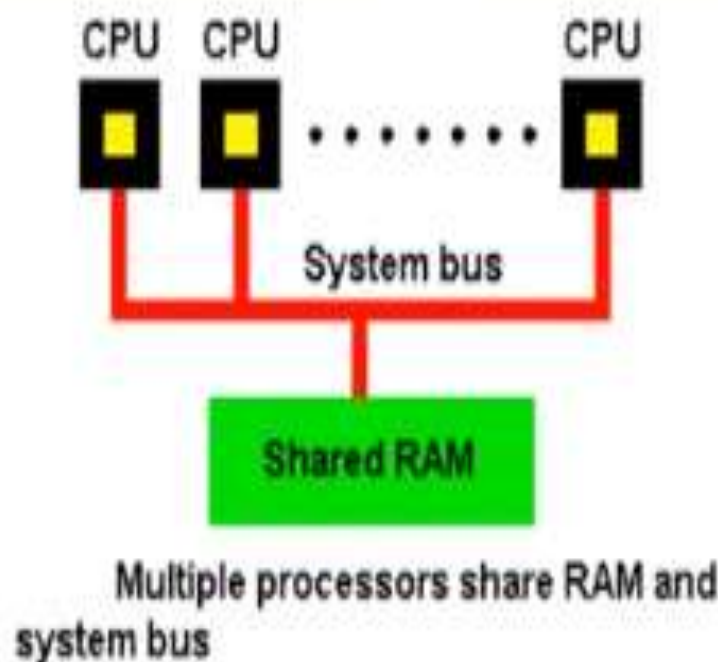
On a single processor system, there is **one main CPU** capable of **executing a general-purpose instruction set**, including instructions from user processes.



COMPUTER SYSTEM ARCHITECTURE

Multiprocessor Systems

Multiprocessor systems have **two or more processors** in **close communication**, sharing the computer bus and sometimes the clock, memory, and peripheral devices.



ADVANTAGES OF MULTIPROCESSOR SYSTEM

Multiprocessor system has following three advantages;

- ❑ Increased throughput.

- ❑ Economy of Scale.

- ❑ Increased reliability.

INCREASED THROUGHPUT

□ By **increasing the number of processors**, we expect to get **more work done in less time**. We only have a certain amount of overhead is incurred in keeping all the parts working correctly.

ECONOMY OF SCALE

- ❑ Multiprocessor systems can **cost less** than equivalent multiple single processor systems, because **they can share peripherals, mass storage, and power supplies**.
- ❑ If several programs operate on the same set of data, it is cheaper to store those data on one disk and to have all the processors share them.

INCREASED RELIABILITY

- ❑ If function can be **distributed** properly among several processors, then failure of one processor will not halt the system, only slow down it.
- ❑ **For example:** If we have ten processor and one fails, then each of the remaining nine processors can pick up a share of the work of the failed processor. Thus entire system runs only 10 percent slower, rather than failing altogether.

MULTIPROGRAMMING

- ❑ Multiprogramming is a **technique used to utilize maximum CPU time** by running multiple programs simultaneously.
- ❑ The execution begins with the first program and continues till an instruction waiting for a peripheral is reached.
- ❑ In simple we can say that to **execute more than one programs at a time** is called multiprogramming.

MULTIPROGRAMMING (CONT.)

The idea is as follows: the operating system keeps several jobs in memory simultaneously (see figure below).



Memory layout for a multiprogramming system

MULTIPROGRAMMING (CONT.)

- ❑ The operating system picks and begins to execute one of the jobs in memory.
- ❑ Eventually, the job may have to wait for some task, such as an I/O operation, to complete.
- ❑ In multi-programmed system, the operating system simply switches to, and executes another job.
- ❑ When that job needs to wait, the CPU switched to another job, and so on.
- ❑ Eventually, first job finishes waiting and gets the CPU back.
- ❑ As long as at least one needs to execute, the CPU is never idle.

DUAL-MODE OPERATION

In order to ensure the proper execution of the operating system, we must be able to distinguish between the **execution** of **operating system code** and **user defined code**.

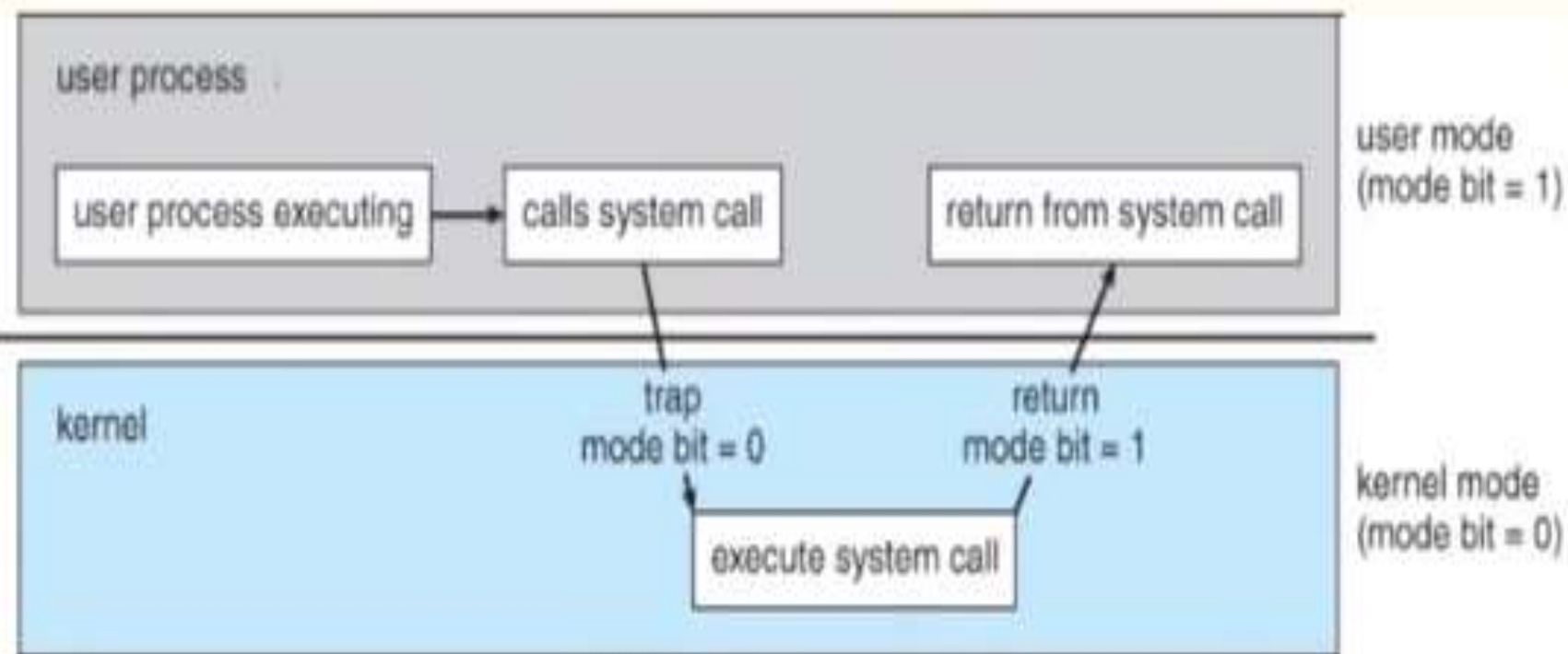
We have two separate modes of operation:

- ❑ **Kernel Mode** (or supervisor mode, system mode or privileged mode)
- ❑ **User mode.**

DUAL-MODE OPERATION (CONT.)

- ❑ At system boot time, the **hardware starts in kernel mode**.
- ❑ Then **operating system is loaded** and starts user application in user mode.
- ❑ Whenever an **interrupt occurs**, the hardware switches from user mode to kernel mode (that is, change the state of the mode bit to 0).
- ❑ Thus whenever the operating system **gains control of the computer**, it is in kernel mode.
- ❑ The system always switches to user mode (by setting the mode bit to 1) before passing the control to a user program.

DUAL-MODE OPERATION (CONT.)



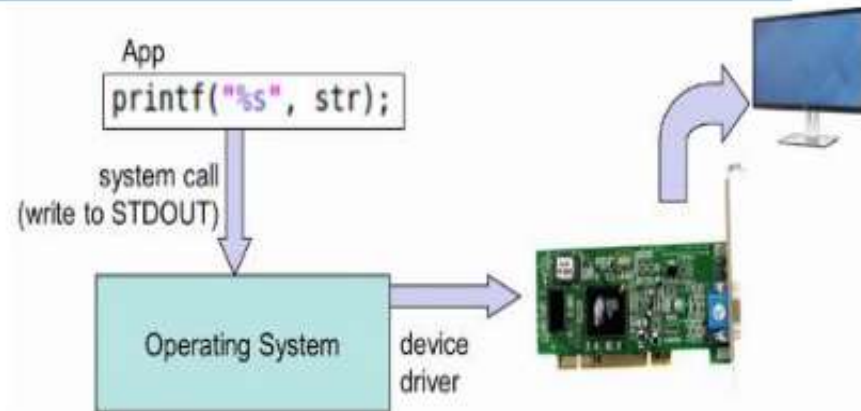
Hardware Abstraction

What is the output of the following program?

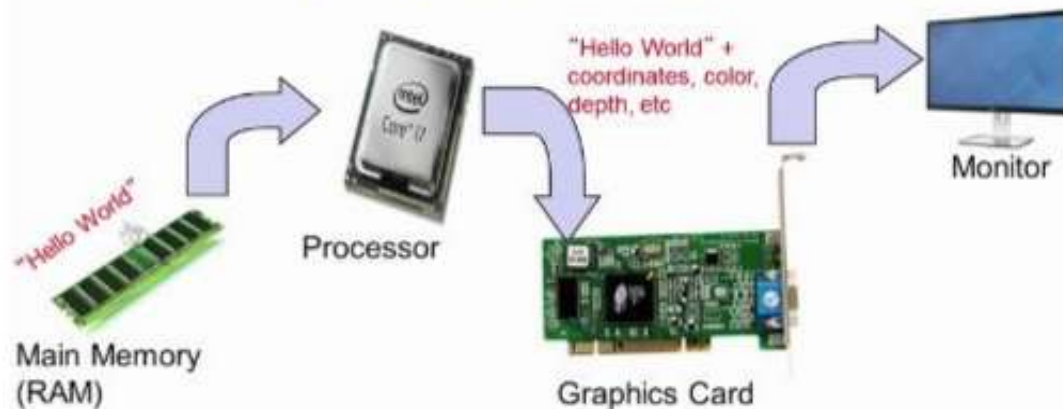
```
#include <stdio.h>

int main(){
    char str[] = "Hello World\n";
    printf("%s", str);
}
```

How is the string displayed on the screen?



Displaying on the Screen



Operating System Services

- Operating systems provide an environment for execution of programs and services (helpful functions) to programs and users
- **User services:**
 - **User interface**
 - ▶ No UI, Command-Line (CLI), Graphics User Interface (GUI), Batch
 - **Program execution** - Loading a program into memory and running it, end execution, either normally or abnormally (indicating error)
 - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device

Operating System Services (Cont.)

■ User services (Cont.):

- **File-system manipulation** - Programs need to read and write files and directories, create and delete them, search them, list file information, permission management.
- **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - ▶ Communications may be via shared memory or through message passing (packets moved by the OS)
- **Error detection** – OS needs to be constantly aware of possible errors
 - ▶ May occur in the CPU and memory hardware, in I/O devices, in user program
 - ▶ For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - ▶ Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services (Cont.)

■ System services:

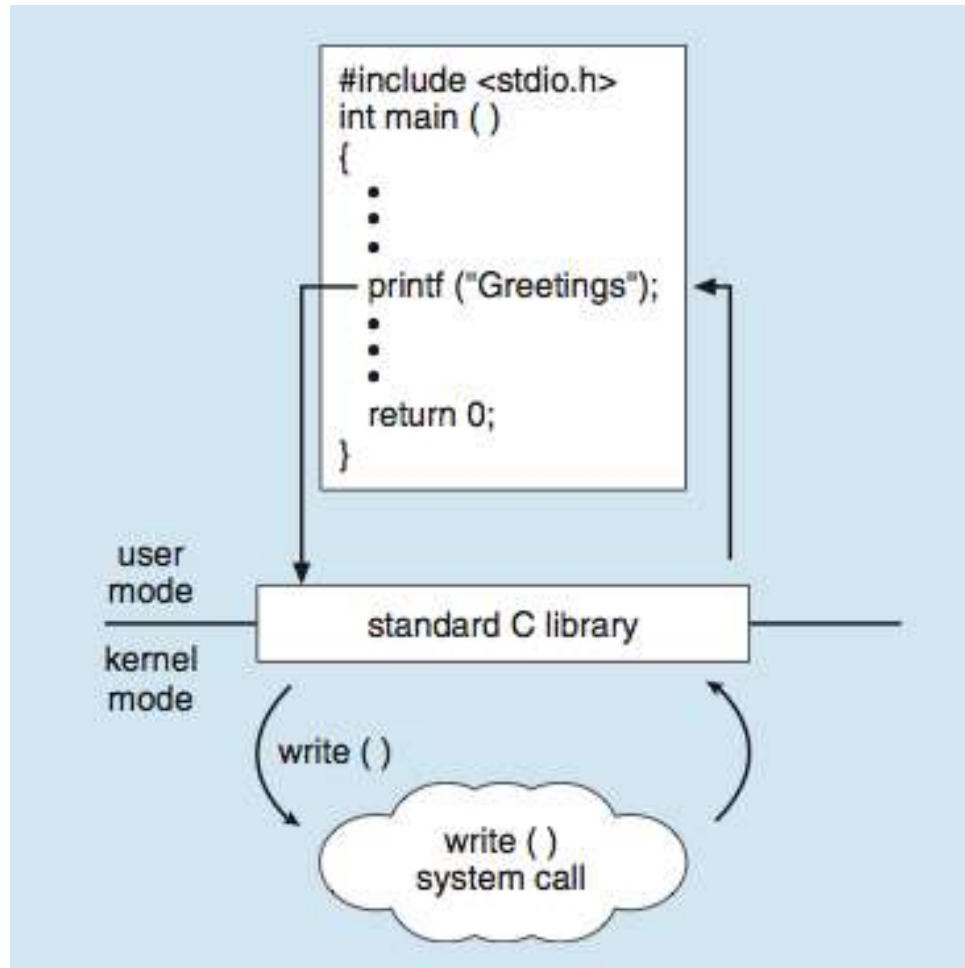
- For ensuring the efficient operation of the system itself via resource sharing
- **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - ▶ Many types of resources - CPU cycles, main memory, file storage, I/O devices.
- **Accounting** - To keep track of which users use how much and what kinds of computer resources
- **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - ▶ **Protection** involves ensuring that all access to system resources is controlled
 - ▶ **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

System Calls

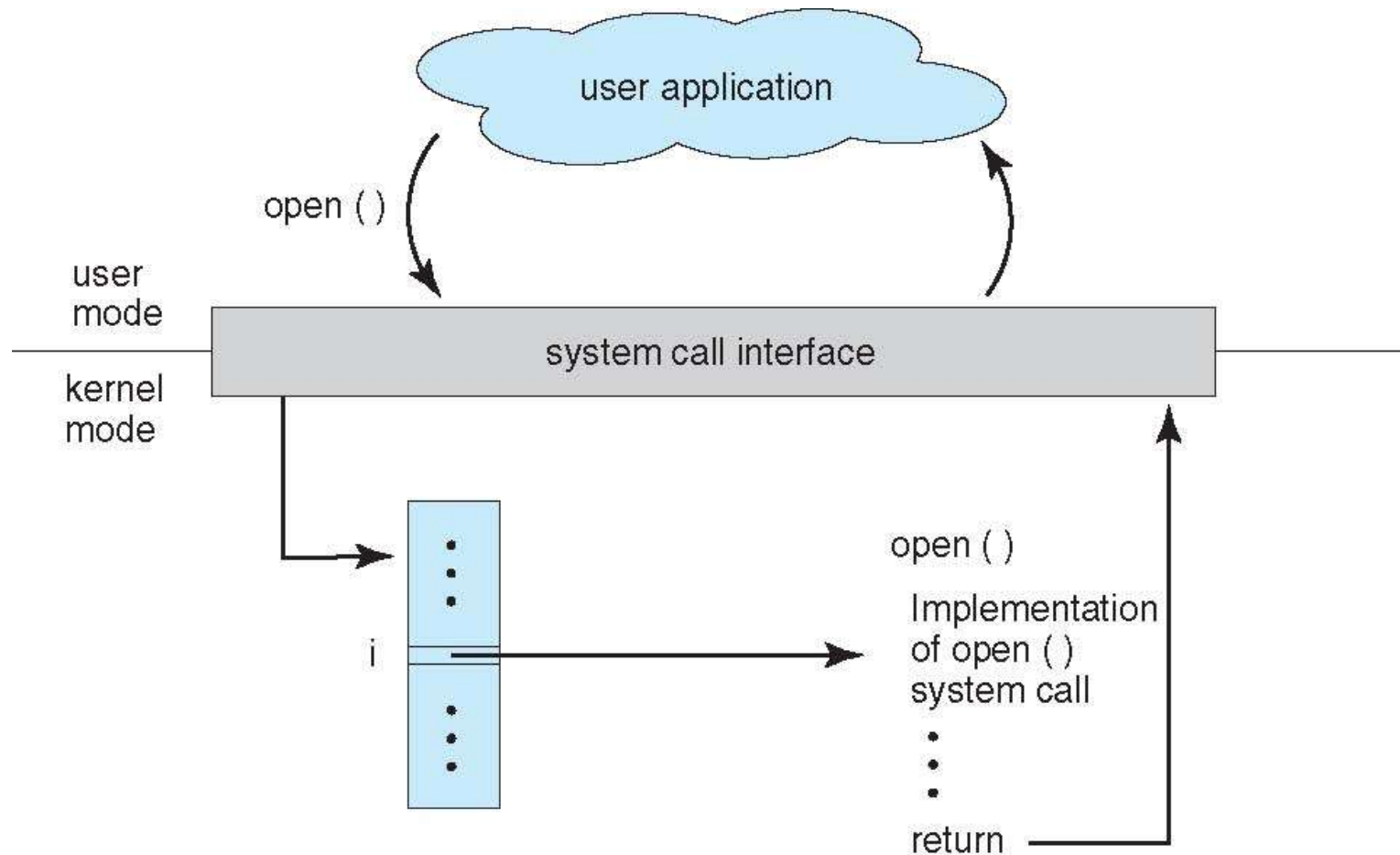
- Systems calls is a request made by the user program to the operating system in order to get any kind of services.
- Typically written in a high-level language (C or C++)

Standard C Library Example

- C program invoking printf() library call, which calls write() system call



System Call – OS Relationship



Types of System Calls

- Process control
 - create process, terminate process
 - end, abort
 - load, execute
 - wait for time
 - allocate and free memory
- File management
 - create file, delete file
 - open, close file
 - read, write
- Device management
 - request device, release device
 - read, write
 - logically attach or detach devices

Types of System Calls

- Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, receive messages
 - transfer status information

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()